Algorithms for Data Science — Revision Sheet (2026)

2 hours · 13 tasks · 180 points · no aids in the room · read this beforehand

Before anything else

⚠ **Heights: use the convention the question prints.** The 2026 papers say **a leaf has height 0** and an **empty subtree is −1**. Your lab sheets say a leaf is 1, which would make every answer one too large. The paper states it in task 6(c) — read that line first.

Order to attempt: 12, 10, 7, 5, 4, 11, then 1, 2, 6, 3, 9, 8, 13. Tasks 5, 7, 10, 11 and 12 are 80 points between them and need execution rather than insight. Start with gradient descent — 15 points of pure recall in about five minutes.

Gradient descent

task 12 · 15 pts

The update rule, with every symbol defined, because that's where the marks sit:

> **$\theta_{t+1} = \theta_t - \eta \cdot \nabla L(\theta_t)$**
> θ_t = current parameters · $\eta > 0$ = learning rate · ∇L = gradient of the loss

Why the minus sign. The gradient points in the direction of steepest *increase* of the loss. We want to decrease it, so we step the opposite way. For small enough η this is guaranteed to reduce the loss locally.

Learning rate. Too small and training crawls — many iterations to get anywhere. Too large and the step overshoots the minimum, so the loss oscillates or diverges.

Convexity. A function is convex if its graph lies at or below the chord joining any two of its points — equivalently, **it has no local minimum other than the global one**. On a convex landscape gradient descent always reaches the global minimum, from any starting point. On a non-convex one it can get stuck in a local minimum.

The numeric part. Both papers use $L(w) = (w - a)^2$, so $\nabla L(w) = 2(w - a)$ and each step is $w \leftarrow w - 2\eta(w - a)$. The distance to a is multiplied by $(1 - 2\eta)$ every step.

Setup	Steps	Behaviour
$(w-3)^2$, $\eta = 0.5$, $w_0 = 0$	$w_1 = 3$, $w_2 = 3$	lands exactly, in one step
$(w-5)^2$, $\eta = 0.25$, $w_0 = 1$	$w_1 = 3$, $w_2 = 4$	halves the gap each step — converges but never quite arrives

Also worth a sentence if asked: **batch** GD uses all n examples per step (stable, expensive), **SGD** uses one (cheap, noisy), **mini-batch** is the practical compromise. Backpropagation is just the chain rule applied layer by layer.

Hash tables

task 10 · 10 pts

Take each key mod the table size and append it to that slot's chain, in insertion order. **Draw every slot, including the empty ones** — the completed table is the answer.

Worst case is Θ(n), when all n keys land in one slot. That happens with input crafted against the hash function, or when every key happens to be congruent modulo the table size.

Operation	Hash table	AVL tree
Search	O(1) expected	O(log n)
Insert	O(1) amortised	O(log n)
Delete	O(1) expected	O(log n)
Sorted output	O(n log n) — sort first	**O(n)** — in-order walk

The hash table wins on average speed. The AVL tree wins on **worst-case guarantees** and on being able to produce **sorted output** at all.

Kruskal / MST

task 7 · 20 pts

Sort the edges by weight, then **repeatedly take the cheapest edge that doesn't create a cycle**. The cycle test is **find–union**: reject an edge exactly when both endpoints are already in the same component.

The table wants three columns per row: the edge, **Add or Reject**, and the components afterwards. Two checks that catch nearly every mistake:

> **1.** You must add exactly **n − 1** edges (five for six vertices).
> **2.** Once every vertex is in one component, **every remaining edge is rejected**.

Θ(m log m), dominated by sorting the edges — find–union is near-constant, Θ(α(n)) amortised.

Dijkstra and BFS

task 5 · 20 pts

⚠ **The 2026 graphs are directed** — relax only the **outgoing** edges of the vertex you just extracted.

Set d[s] = 0 and everything else ∞. Then repeat: **extract the unvisited vertex with the smallest d**, and for each outgoing edge, if d[v] + weight < d[w], overwrite d[w] **and** parent[w]. One table row per extraction.

The thing the question is really testing: **extraction order is not the order vertices were first reached**. If d[3] = 2 and d[2] = 4, vertex 3 comes out first — and going 1→3→2 may then beat the direct edge 1→2.

> **Check every row before moving on:**
> dist[w] = dist[parent[w]] + cost of the edge from parent[w] to w.

Θ((n + m) log n) with a binary heap. Requires non-negative weights.

The BFS part. BFS counts **hops** and ignores weights entirely. They disagree wherever a multi-hop cheap route beats one heavy edge — e.g. BFS gives d[2] = 1 via the direct edge while Dijkstra gives 3 via 1→3→2. Say explicitly that BFS treats every edge as weight 1 and so is only correct on unweighted graphs.

DP: stair climbing

task 4 · 15 pts

With moves of size s_1 and s_2, the number of ways to reach step n is

> **$dp[n] = dp[n - s_1] + dp[n - s_2]$**

Why it holds — write this out, it carries marks: the *last* move to reach step n was either a step of size s_1 (leaving $dp[n - s_1]$ ways of having got there) or of size s_2. The two cases are **disjoint and exhaustive**, so you add them.

dp[0] = 1, not 0 — there is exactly one way to be at the start: don't move. Then fill in the small cases by hand wherever the larger step is impossible.

n	0	1	2	3	4	5	6	7	8
1 or 2	1	1	2	3	5	8	13	21	34
1 or 3	1	1	1	2	3	4	6	9	13

The 1-or-2 row is Fibonacci. For 1-or-3, note dp[2] = 1 because a 3-step can't be used yet. **Θ(n)** time — one left-to-right pass, each entry computed once; Θ(1) space if you keep only the last few values. If the step sizes change, **redo the base cases from scratch** — that's the trap.

The two skeletons

task 11 · 15 pts

Five blanks to fill, then seven scrambled steps to order. Both are pure memorisation.

```
BFS
dist[s] = 0
parent[s] = NONE
q = new Queue;  q.add(s)
while q not empty do
   v = q.extractOldest()
   for every edge (v,w) do
      if dist[w] not yet computed then
         dist[w] = dist[v] + 1
         parent[w] = v
         q.add(w)
```

```
DIJKSTRA
dist[s] = 0;  dist[u] = infinity for u != s
q = new PriorityQueue;  q.add(s, 0)
while q not empty do
   v = q.extractMin()
   for every edge (v,w,c) do
      if dist[w] > dist[v] + c then
         dist[w] = dist[v] + c
         parent[w] = v
         q.add(w, dist[w])
```

Same shape both times: the condition compares, the body assigns, then you push the neighbour. BFS adds 1 and tests "have I seen this yet"; Dijkstra adds the edge weight and tests "can I do better".

Ordering the scrambled steps follows: initialise → build the structure → set up the source → **the loop line** → extract → process neighbours → return. ⚠ **The loop line comes before the body lines it encloses** — that's the only real difficulty, and it's 8 points.

Kruskal's ordering: empty MST → sort edges → initialise find–union → for each edge in order → if the endpoints differ, add and union → otherwise discard → return.

Heaps, BSTs, AVL

task 6 · 15 pts

Heap. Children of index i are 2i and 2i+1; the parent is $\lfloor i/2 \rfloor$. Every parent must be ≥ both children. The answer is the index of the **child**, and the fix is **any value ≤ the parent**. In a 12-element array indices 7–12 are leaves, so there's no lower bound — but ⚠ **if the violating node has children, your new value must also stay ≥ both of them.**

BST. Left subtree < node < right subtree, all the way down. Never restructure; a key already present changes nothing. **Check by reading in-order — it must come out sorted.**

AVL. At every node, $|h(\text{left}) - h(\text{right})| \leq 1$. Write each height in bottom-up, then check node by node — using the convention printed on the paper.

Asymptotics

task 1 · 10 pts

Three expressions to simplify, then one code fragment. Method: kill constant factors, convert each term into a clean shape, keep only the fastest-growing one.

> **$1 < \log n < (\log n)^k < \sqrt{n} < n < n \log n < n^2 < n^3 < 2^n < 3^n < n!$**

When you see	Write
$\log(n^k)$	$k \cdot \log n$, so just $\Theta(\log n)$
$(\sqrt{2})^{\log n}$	$2^{(\log n)/2} =$ **$\sqrt{n}$**
$\sqrt{n} \cdot \log n + \sqrt{n}$	$\sqrt{n}(\log n + 1) = \Theta(\sqrt{n} \log n)$
$n!$ against 3^n	**$n!$ wins**
2^n against n^{1000}	**2^n wins**

Log bases don't matter inside Θ; **exponential bases do** — 3^n is nothing like 2^n.

Counting loops. The whole task is recognising five patterns:

inner loop of constant length (`y = x−10 to x+10`)	**$\Theta(n)$**
`for i=1..n` then `for j=i..n` (triangular)	**$\Theta(n^2)$**
`while y ≥ x` counting down from 10n	**$\Theta(n^2)$**
`while j ≤ n: j = j·2` (doubling)	**$\Theta(\log n)$**, so $\Theta(n \log n)$ nested
innermost `for k = 1 to 10`	contributes nothing

Sums: $1+2+\ldots+n = n(n+1)/2 = \Theta(n^2)$, and $1+2+4+\ldots+2^n = \Theta(2^n)$. **The tell for a logarithm is a loop variable that gets multiplied or divided**, not incremented.

Recurrences

task 2 · 15 pts

First part: read the recurrence off some pseudocode. **Count the recursive calls exactly** — that's where the trap is.

`while i > n−4` starting at i = n	i = n, n−1, n−2, n−3 → **4 calls**
`while i ≤ n−3` starting at i = 1	**n − 3 calls** — depends on n
`for j = 1 to 9`	9 calls
final `for k = 1 to` n^d	$\Theta(n^d)$ of work

So 4 + 12 calls on $\lfloor n/15 \rfloor$ with $\Theta(n^8)$ work gives $T(n) = 16 \cdot T(\lfloor n/15 \rfloor) + \Theta(n^8)$.

⚠ **But (n−3) + 9 = n + 6 calls means the branching factor depends on n, and then the Master theorem does not apply.** Saying that *is* the answer to that variant.

> **Master theorem.** $T(n) = a \cdot T(n/b) + \Theta(n^d)$, with **$c = \log_b a$**
>
> $d < c \rightarrow$ **Case A** $\rightarrow \Theta(n^c)$
>
> $d = c \rightarrow$ **Case B** $\rightarrow \Theta(n^c \log n)$
>
> $d > c \rightarrow$ **Case C** $\rightarrow \Theta(n^d)$

Write out all six things: a, b, f(n), c, the case letter, the result. **c is the power of b that gives a** — powers of 2 run 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024 for k = 1… 10; powers of 3 are 3, 9, 27, 81.

Seen in the papers: $8T(n/2)+n^2 \rightarrow \Theta(n^3)$ case A · $4T(n/2)+n^2 \rightarrow \Theta(n^2 \log n)$ case B · $27T(n/3)+n^2 \rightarrow \Theta(n^3)$ case A · $2T(n/2)+n \rightarrow \Theta(n \log n)$ case B.

Sorting

task 3 · 15 pts

Choosing a sort. The scenario is always many elements with values in a bounded range, so the answer is **counting sort** (or radix), running in $\Theta(n + k)$. Add the punchline: this beats the $\Omega(n \log n)$ bound **because it never compares elements at all**.

Tracing. For MergeSort, show the split levels down to singletons and then the merge levels back up — the marks are for showing both phases. For QuickSort with the last element as pivot, the pivot ends in its final position with everything ≤ it on the left and everything greater on the right; then recurse on each side.

The lower-bound argument — learn this as a paragraph:

> Any comparison-based sort is a **decision tree**: internal nodes are comparisons, leaves are permutations. Every one of the **n!** permutations must be reachable, so the tree has at least n! leaves, so its height is at least $\log_2(n!) =$ **$\Theta(n \log n)$**.
>
> Hence **$\Omega(n \log n)$** comparisons in the worst case — and MergeSort attains it, so MergeSort is optimal.

Variants ask "can we sort in O(n log log n)?" or "in O(n·α(n))?" — the answer is always **no**, since both are o(n log n). Always say **"comparison-based"**, because counting sort is exactly the exception.

	worst	space	stable
insertion	n^2 (n if sorted)	1	yes
merge	n log n	n	yes
quick	**n^2** (avg n log n)	log n	no
heap	n log n	1	no
counting	n + k	n + k	yes
radix	d(n + k)	n + k	yes

Choosing a data structure

task 9 · 15 pts

Three scenarios, five points each, from a list of eight options. Match the keyword:

The scenario says	Answer
retrieve the highest priority; triage; scheduling	**Max-heap / priority queue**
"are these in the same group?"; merging groups	**Find–union**
worst-case O(log n) **and** sorted output	**AVL tree**
O(1) **average**, ordering never needed	**Hash table**
strict first-in-first-out only	**Queue**
static data, binary search, least memory	**Sorted array**
only ever add, never search	**Unsorted array**

The distinguishing facts: a heap supports **no key lookup**; a hash table is $\Theta(n)$ in the **worst** case and cannot give sorted output; an unbalanced BST degrades to $\Theta(n)$; find–union does dynamic connectivity and nothing else.

Write four parts every time: name it, map it onto the scenario, give **time and space** complexity, and say **why each other option loses here** — grouped, not one by one. The fourth part is where most of the marks are and it's the part people skip.

Scenarios already used: ER patients → heap; friend groups and merging bacteria → find–union; leaderboard and airline bookings → AVL; compiler symbol table → hash; HTTP requests → queue.

P and NP

task 8 · 15 pts

P — decision problems solvable deterministically in polynomial time. **NP** — decision problems whose proposed solution (a **certificate**) can be **verified** in polynomial time. **NP-complete** — a problem that is in NP *and* that every NP problem reduces to in polynomial time.

In P: sorting, shortest path with non-negative weights (Dijkstra), minimum spanning tree, binary search, **2-colouring** / **bipartiteness** (BFS), connectivity. **NP-complete:** Hamiltonian cycle, **SAT** (the first, Cook–Levin 1971), knapsack (decision), independent set, vertex cover, **3-colouring**, TSP.

⚠ **2-colouring is in P but 3-colouring is NP-complete; shortest path is in P but Hamiltonian path is NP-complete.** The exam leans on these near-misses.

True statements: if an NP-complete problem is in P then P = NP · $P \subseteq NP$ · every NP problem reduces to any NP-complete one · every NP-complete problem reduces to every other · SAT is NP-complete and was the first proved so.

False: parallel processors solve NP-complete problems in polynomial time · NP-complete problems can only be approximated · all NP problems need exponential time · Hamiltonian cycle is in P.

Algorithm design

task 13 · 10 pts

A skeleton with five blanks, then complexity, then a trace. Both papers use a single pass, so the answer to the complexity part is always **$\Theta(n)$ time and $\Theta(1)$ space** — justify both in a line.

> **Sliding window (fixed k):** new sum = old sum − **w[i−1]** (leaving) + **w[i+k−1]** (entering)

> **Longest run:** if T[i] > T[i−1] extend the run and update the best; otherwise reset with currStart = i and currLen = 1

Variants (longest non-decreasing run, maximum instead of minimum window, longest run of equal values) change only the comparison — the skeleton is identical.

Before you hand in

Heights — did you use the paper's convention? · Kruskal — exactly n−1 edges added? · Dijkstra — does every dist equal the parent's dist plus the edge? · BST — does in-order come out sorted? · Recurrence — is the branching factor constant, and did you say so if not? · Task 9 — did you write why the other structures lose? · dp[0] = 1, not 0 · Master theorem — all six things written out?